

DIAPPOSITIVA 1 / 12

SISTEMA DE OPTIMIZACIÓN DE RUTAS PARA REPARTO URBANO

Proyecto Final de Carrera

Asignatura: Análisis de Algoritmos y Estrategias de Programación

Carrera: Ingeniería de Sistemas Computacionales

Grupo de Trabajo:

- Josué (Expositor 1)
- Ángel (Expositor 2)
- Fabiana (Expositor 3)

■ NOTAS DEL ORADOR — EXPOSITOR: JOSUÉ (Introducción y Negocio)

Buenos días, profesor y compañeros. Hoy nuestro equipo, conformado por Ángel, Fabiana, Gian, Jason y quien les habla, Josué, presentará el proyecto final titulado 'Sistema de Optimización de Rutas para Reparto Urbano'. En esta exposición mostraremos cómo modelar y solucionar un problema crítico de logística urbana utilizando dos enfoques algorítmicos: Fuerza Bruta y un enfoque Voraz. Analizaremos sus complejidades, ventajas y limitaciones técnicas para determinar cuál es la mejor decisión de ingeniería en escenarios del mundo real. Sin más preámbulos, comencemos.

DIAPOSITIVA 2 / 12

PLANTEAMIENTO DEL PROBLEMA: LA ÚLTIMA MILLA

El Reto de la Logística Urbana:

- **Pérdidas Económicas:** La ineficiencia en las rutas de reparto incrementa exponencialmente los costos de combustible, desgaste de flota y tiempos muertos.
- **Crecimiento del E-Commerce:** La alta demanda de entregas a domicilio exige una planificación diaria instantánea de múltiples puntos de distribución.

Modelamiento Matemático:

- Se define como una variante del **Problema del Viajante de Comercio (TSP)**.
- **Objetivo:** Encontrar el circuito de costo mínimo que visite todos los puntos de entrega exactamente una vez y regrese al depósito central de origen.

■ NOTAS DEL ORADOR — EXPOSITOR: JOSUÉ (Introducción y Negocio)

El crecimiento del comercio electrónico ha incrementado drásticamente la demanda de entregas a domicilio, conocidas como la 'última milla'. Las empresas de reparto se enfrentan diariamente al reto de planificar rutas para múltiples destinos. Una mala planificación genera pérdidas significativas de dinero en combustible, desgaste de vehículos y pérdida de tiempo. Matemáticamente, este escenario se modela como el Problema del Viajante de Comercio o TSP. El TSP consiste en encontrar la ruta más corta posible que visite un conjunto de puntos geográficos exactamente una vez y regrese al punto de partida o depósito. Este problema es clasificado como NP-hard, lo que significa que su complejidad escala exponencialmente y requiere estrategias algorítmicas inteligentes.

DIAPOSITIVA 3 / 12

OBJETIVOS Y ALCANCE DEL PROYECTO

Objetivos Principales:

1. Modelar la infraestructura urbana como un **Grafo Ponderado**.
2. Desarrollar e implementar un algoritmo exacto por **Fuerza Bruta** para garantizar el óptimo global en redes pequeñas.
3. Diseñar e implementar un algoritmo heurístico **Voraz (Greedy)** de Vecino Más Cercano para resolver redes de gran escala de forma instantánea.
4. Comparar el desempeño en tiempo de ejecución y costo de ruta de ambas estrategias.

Alcance:

- Pruebas y validaciones experimentales con tamaños de red parametrizables desde 4 hasta 100

■ NOTAS DEL ORADOR — EXPOSITOR: JOSUÉ (Introducción y Negocio)

El objetivo principal de nuestro proyecto es desarrollar y comparar dos enfoques de resolución para el TSP en una red urbana simulada. El primer enfoque es Fuerza Bruta, diseñado para obtener la ruta perfecta (el óptimo global) en instancias pequeñas. El segundo es un Algoritmo Voraz, diseñado para proporcionar soluciones rápidas y de alta calidad para redes grandes, donde la fuerza bruta falla por completo. Nuestro alcance incluye la simulación de coordenadas en un plano bidimensional, la estructuración de un grafo ponderado, la medición de tiempos de ejecución de ambos algoritmos y el análisis del trade-off entre optimalidad y eficiencia computacional.

DIAPOSITIVA 4 / 12

MODELAMIENTO MATEMÁTICO: TEORÍA DE GRAFOS

La ciudad se modela formalmente como un **Grafo Ponderado No Dirigido** $G = (V, E, w)$:

- **Vértices / Nodos (V):** Representan los puntos de entrega de los clientes y el Depósito central de salida ($V = \{v_0, v_1, \dots, v_n\}$).
- **Aristas (E):** Conexiones viales directas entre los puntos ($E \subseteq V \times V$).
- **Función de Costo / Pesos (w):** Asigna a cada arista su distancia euclidiana física o tiempo estimado de tránsito ($w: E \rightarrow \mathbb{R}^+$).

Restricciones del Circuito Hamiltoniano:

1. Partir del nodo inicial 'Depósito'.
2. Visitar cada nodo de entrega exactamente una vez.

■ NOTAS DEL ORADOR — EXPOSITOR: ÁNGEL (Modelado Matemático)

Gracias, Josué. Mi nombre es Ángel y explicaré el modelado matemático. Para resolver este problema, representamos la ciudad como un grafo no dirigido ponderado $G=(V, E, w)$. Los nodos V representan el depósito central y los puntos de entrega de los clientes. Las aristas E representan las conexiones de transporte o calles. Y los pesos w corresponden a la distancia euclidiana entre las coordenadas geográficas de los puntos. La restricción fundamental es encontrar un ciclo Hamiltoniano: el repartidor debe partir del depósito, visitar cada cliente exactamente una vez, y regresar al depósito. El objetivo es minimizar el costo total del recorrido, el cual se calcula sumando los pesos de las aristas seleccionadas.

DIAPOSITIVA 5 / 12

ESTRATEGIAS ALGORÍTMICAS A COMPARAR

1. Enfoque Exacto (Fuerza Bruta):

- Genera sistemáticamente todas las permutaciones posibles de los nodos intermedios a visitar.
- **Garantía:** Encuentra inequívocamente el óptimo global (la ruta más corta absoluta).
- **Limitación:** Computacionalmente intratable para instancias grandes debido al crecimiento de tipo factorial.

2. Enfoque Heurístico (Algoritmo Voraz / Greedy):

- Basado en la heurística del **Vecino Más Cercano (Nearest Neighbor)**.
- **Principio:** En cada nodo actual, elige como siguiente paso el nodo no visitado con menor distancia de conexión.

■ NOTAS DEL ORADOR — EXPOSITOR: ÁNGEL (Modelado Matemático)

Para resolver este modelo, implementamos dos estrategias algorítmicas con filosofías opuestas. Por un lado, la Fuerza Bruta, que evalúa de forma exhaustiva todas las permutaciones posibles de los clientes a visitar. Si bien esto garantiza que siempre encontraremos la ruta con la menor distancia matemática posible, su costo computacional es altísimo y colapsa al superar los 10 nodos. Por otro lado, desarrollamos una estrategia Voraz basada en la heurística del Vecino Más Cercano. Este algoritmo toma decisiones locales e irrevocables en cada paso, moviéndose siempre al punto de entrega no visitado que esté más cerca del actual. Es una técnica altamente escalable y adecuada para producción.

DIAPOSITIVA 6 / 12

PASOS DEL ALGORITMO VORAZ (VECINO MÁS CERCANO)

El flujo lógico del algoritmo voraz se estructura de la siguiente manera:

Paso 1: Inicializar el nodo actual $v_{\text{actual}} = v_{\text{inicio}}$ (Depósito). Marcar como visitado.

Paso 2: Agregar v_{actual} a la lista del recorrido.

Paso 3: Mientras existan nodos de entrega sin visitar en el Grafo:

- Buscar un nodo u no visitado tal que el peso $w(v_{\text{actual}}, u)$ sea el mínimo.
- Acumular el peso: $\text{Costo} = \text{Costo} + w(v_{\text{actual}}, u)$.
- Actualizar nodo actual: $v_{\text{actual}} = u$, y marcar u como visitado.

Paso 4: Cerrar el ciclo: conectar el último nodo visitado con v_{inicio} .

■ NOTAS DEL ORADOR — EXPOSITOR: FABIANA (El Corazón Algorítmico)

Buenos días a todos, soy Fabiana. Les explicaré el corazón algorítmico del enfoque Voraz. Como observan en la diapositiva, el algoritmo sigue un flujo lineal sumamente elegante. Iniciamos en el Depósito y lo marcamos como visitado. Luego, mediante un bucle iterativo que se repite hasta visitar todos los nodos, consultamos los vecinos del nodo actual. Buscamos el nodo vecino no visitado que tenga la menor arista de conexión, sumamos esta distancia a nuestro costo acumulado, nos movemos a ese nodo y lo marcamos como visitado. Al final del bucle, simplemente añadimos la arista de retorno desde el último punto visitado hacia el depósito original. Este proceso evita cualquier tipo de exploración combinatoria hacia atrás (backtracking), lo cual asegura su gran rapidez.

DIAPOSITIVA 7 / 12

ANÁLISIS DE COMPLEJIDAD ASINTÓTICA (BIG O)

Fuerza Bruta: Complejidad Temporal $O(N \cdot N!)$ • Espacial $O(N)$

Excluyendo el origen, existen $(N-1)!$ rutas posibles. Evaluar cada ruta requiere $O(N)$ operaciones de suma. Las operaciones totales crecen de forma factorial:

$$T(N) = N \cdot (N-1)! = N! \implies O(N!)$$

Para $N=20$: 2.43×10^{18} operaciones. Inviabile en la práctica.

Algoritmo Voraz: Complejidad Temporal $O(N^2)$ • Espacial $O(N)$

En el paso k , se evalúan los $N-k$ vecinos restantes para buscar el mínimo. El número total de comparaciones de aristas es la suma aritmética:

$$S(N) = \sum_{k=1}^{N-1} (N-k) = \frac{N(N-1)}{2} = O(N^2)$$

Para $N=20$: 190 comparaciones. Para $N=100$: 4.950 comparaciones (microsegundos).

■ NOTAS DEL ORADOR — EXPOSITOR: FABIANA (El Corazón Algorítmico)

Ahora pasamos al análisis de complejidad, la base teórica de nuestro estudio. El algoritmo de Fuerza Bruta requiere evaluar todas las ordenaciones de los $N-1$ nodos restantes. Esto se traduce en $(N-1)!$ combinaciones. Como verificar el costo de cada camino requiere $O(N)$ operaciones, la complejidad temporal final es de $O(N!)$. Su crecimiento es factorial, por lo que con solo 20 nodos requeriríamos 2.43×10^{18} operaciones, algo que tardaría años en una computadora normal. Por el contrario, el Algoritmo Voraz realiza una búsqueda secuencial en cada paso. Para N nodos, realizamos N búsquedas, donde cada búsqueda examina los nodos restantes. La sumatoria de estas comparaciones nos da un orden de complejidad temporal de $O(N^2)$. Para 20 nodos, solo toma unas 190 operaciones básicas, resolviéndose en microsegundos. Ambos algoritmos tienen complejidad espacial $O(N)$ para almacenar el grafo y las listas de control en memoria.

DIAPOSITIVA 8 / 12

IMPLEMENTACIÓN E INGENIERÍA DE SOFTWARE

Decisiones de Implementación:

- **Lenguaje:** Python 3.8+ utilizando únicamente su biblioteca estándar (alta portabilidad e independencia de dependencias de terceros).
- **Robustez:** Tipado fuerte estático con la librería `typing` para prevenir bugs en la manipulación de datos del grafo.

Estructura del Proyecto en Disco:

```
proyecto-final/  
  README.md (Documentación técnica y análisis)  
  data/  
    datos_entrada.csv (Persistencia del grafo)  
    src/
```

■ NOTAS DEL ORADOR — EXPOSITOR: GIAN (Ingeniería y Comparación)

Gracias, Fabiana. Mi nombre es Gian e iniciaré explicando las decisiones de ingeniería de software. Implementamos la solución en Python utilizando exclusivamente su biblioteca estándar, lo que garantiza la portabilidad del sistema en cualquier entorno sin requerir complejas instalaciones previas. Diseñamos el software bajo un enfoque modular y fuertemente tipado para evitar errores de tipo en tiempo de ejecución. La estructura del repositorio separa limpiamente la lógica algorítmica en 'algoritmos.py', la lógica de presentación y flujo interactivo en 'main.py', y las fuentes de datos en la carpeta 'data/'. Esto facilita enormemente el mantenimiento del código y cumple con las buenas prácticas de desarrollo sugeridas en la guía de proyectos.

DIAPOSITIVA 9 / 12

CASOS DE PRUEBA Y GENERACIÓN DE DATOS

Preparación Previa de Datos de Entrada:

- El sistema cuenta con una función para autogenerar **datos_entrada.csv** simulados, evitando pérdidas de tiempo en la exposición.
- **Plano Cartesiano 2D:** Genera coordenadas aleatorias $(x, y) \in [0, 100]^2$ para cada uno de los nodos requeridos.
- **Desigualdad Triangular:** Las distancias se calculan bajo la métrica de distancia euclidiana regular:

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- Esto asegura consistencia geométrica realista en el grafo completo generado.

■ NOTAS DEL ORADOR — EXPOSITOR: GIAN (Ingeniería y Comparación)

Para probar y validar los algoritmos, desarrollamos un módulo de simulación de datos en `main.py`. Este módulo simula la distribución de clientes en un plano cartesiano de 100 por 100 kilómetros, asignando coordenadas aleatorias. Luego, calcula la distancia euclidiana entre cada par de puntos y escribe estas distancias en el archivo `'datos_entrada.csv'`. Al usar distancias geométricas reales, garantizamos de forma natural la desigualdad triangular, la cual establece que el camino directo entre dos puntos es siempre menor o igual a pasar por un punto intermedio. Esto simula fielmente la geografía urbana y nos permite cargar grafos completos y consistentes para las pruebas.

DIAPOSITIVA 10 / 12

RESULTADOS EXPERIMENTALES Y COMPARATIVA

Nodos (N)	Costo (FB)	Tiempo (FB)	Costo (Voraz)	Tiempo (Voraz)
5	175.12 km	0.00005 s	191.08 km	0.00001 s
7	176.23 km	0.00106 s	209.66 km	0.00001 s
10	239.62 km	0.97054 s	278.15 km	0.00003 s
20	No viable	N/A (Excesivo)	595.58 km	0.00007 s
100	No viable	N/A (Excesivo)	1014.29 km	0.00146 s

■ NOTAS DEL ORADOR — EXPOSITOR: GIAN (Ingeniería y Comparación)

En la diapositiva podemos observar la tabla comparativa que genera el sistema de manera automática. Para un tamaño pequeño de 5 a 7 nodos, la diferencia en tiempo es despreciable y el algoritmo Voraz encuentra un costo muy cercano al óptimo real. Sin embargo, al llegar a 10 nodos, Fuerza Bruta empieza a experimentar la barrera factorial tardando casi un segundo completo (0.97 segundos), mientras que el algoritmo Voraz resuelve la ruta en apenas 29 microsegundos. Para 20 nodos o más, la Fuerza Bruta se vuelve completamente inviable. En contraste, el algoritmo Voraz es capaz de resolver una red masiva de 100 puntos de entrega en solo 1.4 milisegundos. Esta tabla demuestra empíricamente la efectividad de las heurísticas en problemas de alta complejidad combinatoria.

DIAPOSITIVA 11 / 12

DEMOSTRACIÓN FUNCIONAL EN VIVO

Guión de la Demostración Práctica:

1. **Ejecutar** ``python main.py`` desde la consola de comandos de Windows.
2. **Generar Datos (Opción 1):** Simular un conjunto parametrizado de puntos de entrega (ej. 15 nodos) para actualizar el CSV.
3. **Calcular Rutas (Opción 2):** Cargar el grafo y ejecutar los algoritmos. Mostrar la omisión automática de Fuerza Bruta para evitar cuelgues.
4. **Prueba Comparativa (Opción 3):** Correr la simulación masiva automatizada y visualizar la tabla de tiempos y costos en tiempo real.
5. **Grafo Manual (Opción 4):** Registrar nodos y aristas bajo demanda para verificar casos teóricos de prueba.

■ NOTAS DEL ORADOR — EXPOSITOR: JASON (Demo y Cierre)

Hola a todos, soy Jason. En este momento de la exposición, pasaré a compartir mi pantalla para realizar la demostración funcional en vivo de nuestro sistema. Ejecutaré el script principal en la terminal. Como pueden observar, se despliega un menú interactivo. Primero, seleccionaré la opción 1 para generar un mapa simulado con 15 puntos de entrega. Seguidamente, cargaré este archivo con la opción 2, estableciendo el Depósito como punto de inicio; aquí veremos cómo el sistema autodetecta que al ser 15 nodos, debe omitir la fuerza bruta para evitar congelamientos. Finalmente, ejecutaré la opción 3 para que el profesor observe la tabla de comparación directa en la consola. La demostración valida que el software es robusto y de grado de producción.

DIAPOSITIVA 12 / 12

CONCLUSIONES Y CIERRE

Conclusiones Clave del Proyecto:

- 1. Trade-off de Diseño de Algoritmos:** El algoritmo Voraz es la mejor decisión de ingeniería para la distribución urbana. El 'gap' de error en la distancia total (heurística) es mínimo frente al ahorro masivo en tiempo de procesamiento.
- 2. Inviabilidad Exponencial/Factorial:** Queda demostrada teórica y empíricamente la inviabilidad de Fuerza Bruta ($O(N!)$) para la logística real a gran escala.
- 3. Arquitectura Escalable:** El diseño modular y desacoplado de la clase `Grafo` facilita incorporar heurísticas avanzadas en el futuro (ej. Algoritmos Genéticos, Colonias de Hormigas).

¡Muchas gracias por su atención! Quedamos abiertos a sus preguntas.

■ NOTAS DEL ORADOR — EXPOSITOR: JASON (Demo y Cierre)

Para cerrar nuestra presentación, planteamos tres conclusiones clave. En primer lugar, confirmamos que el Algoritmo Voraz es la mejor opción práctica para la toma de decisiones en logística urbana. Aunque no garantiza la ruta más corta absoluta, el margen de error promedio es muy pequeño frente a la inmensa ganancia de velocidad, resolviendo en milisegundos lo que a la Fuerza Bruta le tomaría años. En segundo lugar, se validó de forma práctica la inviabilidad de los algoritmos factoriales para problemas reales de gran escala. Finalmente, la modularidad de nuestro código en Python deja la arquitectura abierta para implementar heurísticas más complejas en el futuro, como Algoritmos Genéticos o Programación Dinámica. Con esto concluimos nuestra exposición. Quedamos a su disposición para cualquier pregunta o aclaración técnica del profesor. ¡Muchas gracias por su atención!